

Training Tables Specification

Overview

This document specifies the format and content of all 12 training tables used in the BDI training system.

File Format

Header Structure

All training table files begin with a 64-byte header:

```
struct table_header {
    uint32_t magic;           // Magic number: 0x42444954 ('BDIT')
    uint32_t version;         // Format version: 1
    uint32_t entry_type;      // Type of entries in this file
    uint32_t num_entries;     // Number of training entries
    uint64_t total_size;      // Total file size in bytes
    uint32_t checksum;        // CRC32 checksum of entire file
    uint32_t reserved[10];    // Reserved for future use (zeros)
};
```

Entry Structure

Each training entry has the following structure:

```
struct training_entry {
    uint32_t entry_type;      // Type of training entry (0-11)
    uint32_t difficulty;      // Difficulty level (0-7, maps to phases)
    uint32_t input_size;      // Size of input data in bytes
    uint32_t output_size;     // Size of expected output in bytes
    uint8_t data[];           // Variable-length input and output data
};
```

The `data` field contains:

1. Input data (input_size bytes)
2. Expected output data (output_size bytes)

Entry Types

Math Tables

0: ENTRY_MATH_ARITHMETIC

File: `math_arithmetic.dat`

Operations:

- Addition: input=[a, b], output=[a+b]
- Subtraction: input=[a, b], output=[a-b]
- Multiplication: input=[a, b], output=[a*b]

- Division: input=[a, b], output=[quotient, remainder]
- Exponentiation: input=[base, exp], output=[result]

Coverage:

- Addition/Subtraction: 0-1000
- Multiplication: 0-500
- Division: dividend 0-1000, divisor 1-100
- Exponentiation: base 0-20, exp 0-10

Expected Entries: ~1,500,000

1: ENTRY_MATH_MODULAR

File: `math_modular.dat`

Operations:

- Modular addition: input=[a, b, mod], output=[(a+b) % mod]
- Modular multiplication: input=[a, b, mod], output=[(a*b) % mod]
- Modular exponentiation: input=[base, exp, mod], output=[result]

Moduli: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47

Expected Entries: ~200,000

2: ENTRY_MATH_NUMBER_THEORY

File: `math_number_theory.dat`

Operations:

- GCD: input=[a, b], output=[gcd(a,b)]
- LCM: input=[a, b], output=[lcm(a,b)]
- Coprimality: input=[a, b], output=[1 if coprime, 0 otherwise]
- Primality: input=[n], output=[1 if prime, 0 otherwise]

Coverage:

- GCD: 0-1000 for both inputs
- LCM: 1-500 for both inputs
- Coprimality: 1-1000 for both inputs
- Primality: 0-10000

Expected Entries: ~2,500,000

3: ENTRY_MATH_SEQUENCES

File: `math_sequences.dat`

Sequences:

- Fibonacci: input=[n], output=[fib(n)] for n=0-49
- Primes: input=[n], output=[nth prime] for n=0-999
- Perfect squares: input=[n], output=[n²] for n=0-1000
- Triangular: input=[n], output=[n(n+1)/2] for n=0-500
- Factorials: input=[n], output=[n!] for n=0-20
- Catalan: input=[n], output=[catalan(n)] for n=0-30

Expected Entries: ~2,600

Logic Tables

4: ENTRY_LOGIC_BOOLEAN

File: `logic_boolean.dat`

Operations:

- Unary: NOT
- Binary: AND, OR, XOR, NAND, NOR, XNOR, IMPLIES, IFF
- Ternary: Compound expressions with 3 variables
- Quaternary: Compound expressions with 4 variables

Format:

- Unary: input=[op_code, a], output=[result]
- Binary: input=[op_code, a, b], output=[result]
- Ternary: input=[op_code, a, b, c], output=[result]
- Quaternary: input=[op_code, a, b, c, d], output=[result]

Expected Entries: ~100

5: ENTRY_LOGIC_THREE_VALUED

File: `logic_three_valued.dat`

Values: 0=false, 1=true, 2=unknown

Operations:

- NOT: input=[op_code, a], output=[result]
- AND, OR, IMPLIES: input=[op_code, a, b], output=[result]
- Compound: input=[op_code, a, b, c], output=[result]

Expected Entries: ~100

6: ENTRY_LOGIC_PROPOSITIONAL

File: `logic_propositional.dat`

Patterns:

- Tautologies: Always true expressions
- Contradictions: Always false expressions
- Equivalences: Logically equivalent expressions (De Morgan's laws, etc.)
- Contingencies: Sometimes true, sometimes false

Format: input=[pattern_code, variables...], output=[result or equivalence]

Expected Entries: ~50

Language Tables

7: ENTRY_LANGUAGE_NGRAMS

File: `language_ngrams.dat`

N-gram Types:

- Unigrams: Single characters with frequency
- Bigrams: 2-character sequences
- Trigrams: 3-character sequences
- 4-grams: 4-character sequences
- 5-grams: 5-character sequences (limited)

Unicode Ranges:

- Basic Latin (U+0020-U+007E)
- Latin-1 Supplement (U+00A0-U+00FF)
- Latin Extended-A (U+0100-U+017F)
- Greek (U+0370-U+03FF)
- Cyrillic (U+0400-U+04FF)
- CJK samples (U+4E00-U+4EFF)
- Hiragana (U+3040-U+309F)
- Katakana (U+30A0-U+30FF)
- Arabic (U+0600-U+06FF)
- Devanagari (U+0900-U+097F)

Format: input=[codepoint(s)], output=[frequency]

Expected Entries: ~100,000

8: ENTRY_LANGUAGE_SYNTAX

File: `language_syntax.dat`

Pattern Types:

- Case patterns: uppercase/lowercase/titlecase transitions
- Script transitions: Latin→CJK, Latin→Cyrillic, etc.
- Grapheme clusters: base + combining characters
- Grammatical patterns: POS tag sequences

Format: input=[pattern_type, data...], output=[validity or pattern_id]

Expected Entries: ~1,000

Code Tables**9: ENTRY_CODE_AST**

File: `code_ast.dat`

AST Node Types:

- Literals: Numbers, strings, booleans
- Variables: Identifiers
- Binary operations: +, -, *, /, %, &, |, ^, <, >, =
- Unary operations: -, !, ~, ++, -
- Function calls: With 0-5 arguments
- Control flow: if, while, for
- Assignments: Variable = expression
- Returns: return expression

Format: input=[node_type, operands...], output=[validity]

Expected Entries: ~2,000

10: ENTRY_CODE_IDIOMS

File: `code_idioms.dat`

Idiom Types:

- Loop idioms: for(i=0; i<n; i++), while, do-while
- Conditional idioms: if-else chains, switch statements

- Error handling: try-catch, error codes
- Resource management: RAII patterns

Format: input=[idiom_type, parameters...], output=[validity]

Expected Entries: ~1,000

11: ENTRY_CODE_BUGS

File: code_bugs.dat

Bug Types:

- Off-by-one errors: Loop bounds
- Null pointer dereference: Missing null checks
- Buffer overflow: Array bounds violations
- Memory leaks: Missing free/delete
- Type mismatches: Incompatible types
- Logic errors: Incorrect conditions

Format: input=[bug_type, code_pattern...], output=[has_bug: 0 or 1]

Expected Entries: ~1,200

Difficulty Levels

Each entry has a difficulty level (0-7) that maps to curriculum phases:

Level	Phase	Description
0	Foundations	Simplest examples
1	Elementary	Basic complexity
2	Intermediate	Moderate complexity
3	Advanced	Challenging examples
4	Expert	Complex patterns
5	Master	Advanced techniques
6	Virtuoso	Expert-level problems
7	Transcendent	Research-level challenges

Data Validation

Checksum Calculation

The CRC32 checksum is calculated over the entire file excluding the checksum field itself:

```
uint32_t calculate_checksum(const void *data, size_t size) {
    // CRC32 with polynomial 0xEDB88320
    // Standard implementation
}
```

Validation Checks

1. **Magic Number:** Must be 0x42444954 ('BDIT')
2. **Version:** Must be 1
3. **Entry Count:** Must match actual number of entries
4. **File Size:** Must match total_size field
5. **Checksum:** Must match calculated CRC32
6. **Entry Integrity:** All entries must be well-formed

File Size Estimates

File	Estimated Size	Entries
math_arithmetic.dat	50-80 MB	~1,500,000
math_modular.dat	10-15 MB	~200,000
math_number_theory.dat	80-100 MB	~2,500,000
math_sequences.dat	100-150 KB	~2,600
logic_boolean.dat	5-10 KB	~100
logic_three_valued.dat	5-10 KB	~100
logic_propositional.dat	2-5 KB	~50
language_ngrams.dat	50-100 MB	~100,000
language_syntax.dat	50-100 KB	~1,000
code_ast.dat	100-200 KB	~2,000
code_idioms.dat	50-100 KB	~1,000
code_bugs.dat	50-100 KB	~1,200
Total	~300-500 MB	~4,200,000

Usage Examples

Reading a Training Table

```
#include "training_tables.h"

int fd = open("math_arithmetic.dat", O_RDONLY);
table_header_t header;
table_read_header(fd, &header);

training_entry_t *entry;
while (table_read_entry(fd, &entry) > 0) {
    // Process entry
    uint32_t *input = (uint32_t *)entry->data;
    uint32_t *output = (uint32_t *) (entry->data + entry->input_size);

    // Use input and output

    free(entry);
}

close(fd);
```

Validating a Table

```
if (table_validate_file("math_arithmetic.dat") == 0) {
    printf("Table is valid\n");
} else {
    printf("Table validation failed\n");
}
```

Extensibility

The format supports future extensions through:

1. **Version Field:** Can increment for format changes
2. **Reserved Fields:** 40 bytes reserved in header
3. **Entry Types:** Can add new types (12-255)
4. **Variable Data:** Flexible data field supports any content

References

- BDI Training System Documentation
- Curriculum Design Document
- Unicode 17.0.0 Standard
- CRC32 Algorithm (IEEE 802.3)